МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
« МИРЭА — Российский технологический университет»
РТУ МИРЭА

Институт Информационных технологий
Кафедра Вычислительной техники

КУРСОВАЯ РАБОТА

по дисциплине «Объектно-ориентированное программирование»

Тема курсовой работы: К_1 Расстановка символов

Студент группы _ИНБО-06-19_ Громов Н.Д._ _____

(учебная группа, фамилия, имя отчество, студента) (подпись студента)

Руководитель курсовой работы _Ассистент_ Рогонова О.Н._ _____

(должность, звание, ученая степень) (подпись руководителя)

Работа представлена к защите «30» мая 2020 г.

Допущен к защите «30» мая 2020 г.

Москва 2020

Постановка задачи

Дано поле 10 x 10 позиций. Нумерация позиций по горизонтали и по вертикали начинается с 1. Поле 10 x 10 формируется в файле field.txt и первоначально все позиции заполняются 8.
На вход подаются последовательно данные: координата позиции и символ для отметки. Координата позиции задается посредством пары целых чисел от 1 до 10. Первое число номер строки, второе номер столбца.
Начало координат верхний левый угол поля. Символ для отметки принадлежит латинскому алфавиту.
Таких троек данных может быть множество. После получения очередной тройки **выдается сигнал** с текстом, содержащим координату и символ латинского алфавита.
Если координата равна (0, 0), программа завершает работу.
Первая строка входных данных не содержит (0, 0).

Использовать объекты:
1. Для установки позиции в файле согласно координате. Объект **выдает сигнал** об установке позиции или выводит в конце файла сообщение об ошибке.
2. Для записи символа в файле в установленную позицию. Объект пишет символ в установленной позиции или выводит в конце файла сообщение об ошибке.
3. Для вывода результата из файла на консоль. Все сообщения об ошибках пишутся построчно, с новой строки. Первое сообщение выводится с 11 строки (после поля 10 x 10).

Написать программу, реализующую следующий алгоритм:
1. Создание файла и формирование исходного содержания.
2. Ввод первой тройки данных.
3. Начало цикла.
    3.1. Выдача сигнала о вводе данных.
    3.2. Ввод очередной тройки данных и выдача сигнала.
4. Завершение цикла, если введена координата (0, 0).
5. Вывод на консоль содержимого файла.

Описание входных данных

Построчно множество координат и символ:
«целое число»«целое число»«символ»

Описание выходных данных

Десять строк по десять символов в каждой, согласно сформированному в файле полю.

Если какая-либо координата не принадлежит интервалу [1, 10] то выводится сообщение: Coordinate is wrong («значение», «значение»)

Если символ не принадлежит латинскому алфавиту: Not a letter of the Latin alphabet: «символ»

Метод решения

Стандартные потоки ввода/вывода cin/cout

Билиотека vector для работы с векторами

Библиотека string для работы со строками

Библиотека fstream для работы с файлами

Билиотека sstream для работы с строковыми потоками

Структура иерархии объектов

Root

FirstObject

SecondObject

ThirdObject

Структура con

Описание структуры con:

Свойства:

signal psignal - указатель на метод

tarobject - указатель на объект
handler phandler - указатель на метод

Объект класса Base

Описание класса Base:

Свойства:

Структура con

_parent, тип указатель на объект
string _name
int ReadCheck
vector children, тип указатель на объект
vector connections, тип указатель на структуру

GetName()
SetName(string name) - устанавливает имя объекта

Методы:

GetName() - возвращает имя объекта
SetName(string name) - устанавливает имя объекта
AddChild(Base* child) - добавляет потомка в список наследников
SetStatus(int Check) - устанавливает статус объекта
SetConnect(signal psignal, Base* pobject, handler phandler) - установка связи между двумя объектами
DelConnect(signal psignal, Base* pobject, handler phandler) - удаление связи между двумя объектами
EmitSign(signal psignal, string mess) - выдача сигнала объектом

методы из КР:

FoundObject(string name, bool flag)

GetPointTo(string name)

Print(int counter)

Объект класса Root

Описание класса Root:

Класс открыто наследует Base

Свойства:

Все свойства класса Base

Методы:

Все методы класса Base

input_create() - содает и размещает объекты на дереве
start_app() - запускает приложение

Signal(string &mess) - метод сигнала объекта

Объект класса SecDer

Описание класса SecDer:

Класс открыто наследует Base

Свойства:

file1 - тип указатель на объект

Все свойства класса Base

Методы:

SetStream(fstream* file)

Все методы класса Base

Signal(string &mess) - метод сигнала объекта

Handler(string mess) - метод обработчика объекта

Объект класса ThirDer

Описание класса ThirDer:

Класс открыто наследует Base

Свойства:

file1 - тип указатель на объект

Все свойства класса Base

Методы:

Все методы класса Base

SetStream(fstream* file)

Signal(string &mess) - метод сигнала объекта Handler_SetSym(string mess) - метод обработчика, который записывает значение по координате

Объект класса FourDer

Описание класса FourDer:

Класс открыто наследует класс Base

Свойства:

file1 - тип указатель на объект

Все свойства класса Base

Методы:

Все методы класса Base

void OutpFile() - вывод содержимого файла

SetStream(fstream* file)

Описание алгоритма

Основная программа

№Шага	Предикат	Действие	№Перехода

1		Root root(nullptr)
		
		root.input_create();
		
		return root.start_app();
			∅

~Base()

№Шага	Предикат	Действие	№Перехода
1		int i=0	2
2	i<childen.size()	delete this->children[i]	2
		children.clear() i=0	3
3	i<connections[i]	delete this->connections[i]	**3**
		connections.clear()	∅

Base(Base* parent)

№Шага	Предикат	Действие	№Перехода
1		_parent=parent	2
2	_parent!=nullptr	parent->AddChild(this)	∅
			∅

Base(string name, Base* parent)

№Шага	Предикат	Действие	№Перехода
1		_parent=parent _name = name	2
2	_parent!=nullptr	parent->AddChild(this)	∅
			∅

GetName()

№Шага	Предикат	Действие	№Перехода
1		return _name	∅

AddChild(Base* child)

№Шага	Предикат	Действие	№перехода
1		children.push_back(child)	∅

SetStatuc(int Check)

№Шага	Предикат	Действие	№Перехода
1		ReadCheck = Check	∅

SetName(string name)

№Шага	Предикат	Действие	№Перехода
1		_name = move(name)	

SetConnect(signal psignal, Base *pobject, handler phandler)

№Шага	Предикат	Действие	№Перехода
1		int i=0	2
2	i<connections.size()		3
			4
3	(connections[i]->psignal==psignal) && (connections[i]->tarobject==pobject) && (connections[i]->phandler==phandler)	return	∅
	иначе	i++	2

4		con* pntr = new con();
		pntr->psignal = psignal;
		pntr->tarobject = pobject;
		pntr->phandler = phandler;
		connections.push_back(pntr);
			∅

DelConnect(signal psignal, Base *pobject, handler phandler)

№Шага	Предикат	Действие	№перехода
1		int i=0	2
2	i<connections.size()		3
			∅
3	((connections[i]->psignal)==psignal) && ((connections[i]->tarobject)==pobject) && ((connections[i]->phandler)==phandler)	connections.erase(connections.begin()+i); return;	∅
	иначе	i++	2

EmitSign(signal psignal, string mess)

№Шага	Предикат	Действие	№Перехода
1		int i=0	2
2	i<connections.size()		3
			∅
3	((connections[i]->psignal) == psignal)	(((connections[i])->tarobject)->*(connections[i]->phandler))(mess);	2

		i++	
	иначе	i++	2

input_create()

№Шага	Предикат	Действие	№Перехода
1		SecDer* first = new SecDer("FirstObject", this); ThirDer* second = new ThirDer("SecondObject", this); FourDer* third = new FourDer("ThirdObject", this); string buff = "8888888888"; ofstream fileout("field.txt"); int i=0	2
2	i<9	fileout<<buff<<endl	2

		fileout<<buff;
		fileout.close();
		fstream* file1 = new fstream("field.txt");
		(first)->SetStream(file1);
		(second)->SetStream(file1);
		third->SetStream(file1);
		this->SetConnect(SIGNAL_D (Root::Signal), first, HANDLER_D (SecDer::Handler));
		first->SetConnect(SIGNAL_D (SecDer::Signal), second, HANDLER_D (ThirDer::Handler_SetSym));
			∅

start_app()

№Шага	Предикат	Действие	№перехода
1		Base* first = this->children[0], *second = this->children[1]; Base* third = this->children[2]; int CoordinX, CoordinY; char symbol; cin>>CoordinX>>CoordinY;	2

2	(CoordinX != 0 && CoordinY !=0)	cin>>symbol;
		
		string buff1 = to_string(CoordinX);
		string buff2 = to_string(CoordinY);
		
		string mess = buff1+" "+buff2+" "+symbol;
		first->EmitSign(SIGNAL_D (Root::Signal), CoordinX, CoordinY, symbol);
			2

		cin>>CoordinX>>CoordinY;	
		static_cast<FourDer*>(third)third->OutpFile() return 0	∅

Handler(string mess)

№Шага	предикат	Действие	№Перехода
1		stringstream ss; int x,y; char sym; ss<<mess; ss>>x; ss>>y; ss>>sym; mess = sym;	2

2	(x>=1) && (x<=10) &&
	(y>=1) && (y<=10)
		file1->seekp(10*(x-1)+(x-1), file1->beg);
		file1->seekp(y-1, file1->cur);
		this->EmitSign(SIGNAL_D
		(SecDer::Signal), mess);
			∅

	иначе	string buff1 = to_string(x);
		string buff2 = to_string(y);
		string buff = "Coordinate is wrong (" +
		buff1 + ", " + buff2 + ")";
		file1->seekp(0, file1->end);
		(*file1)<<endl<<buff;
			∅

SetStatus(fstream* file)

№Шага	Предикат	Действие	№Перехода
1		file1=file	∅

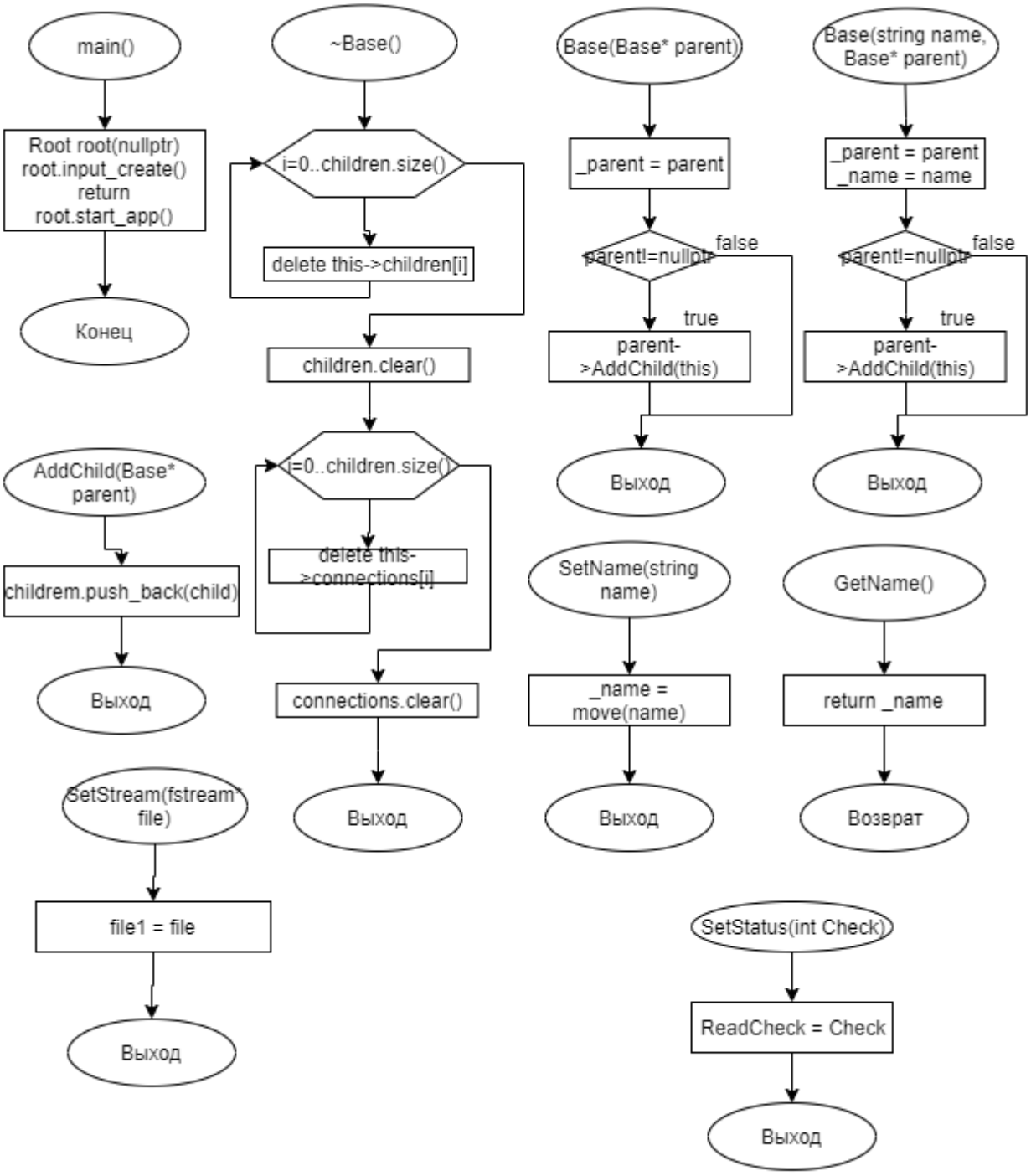
Handler_SetSym(string mess)

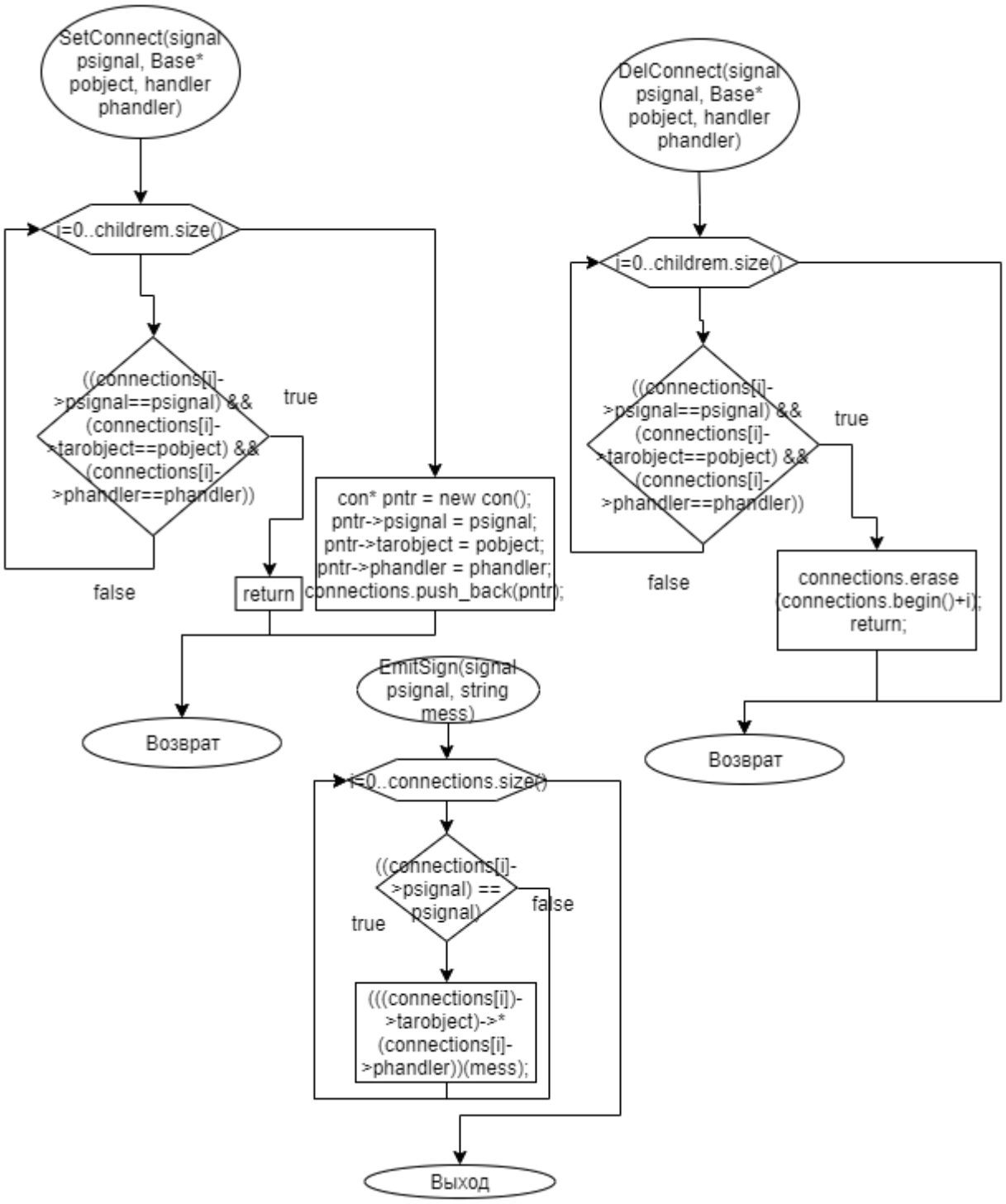
№Шага	Предикат	Действие	№Перехода
1		stringstream ss; char sym; ss<<mess; ss>>sym	2
2	(sym>='A' && sym<='Z') || (sym>='a' && sym<='z')	(*field1)<<sym	∅
	иначе	file1->seekp(0, file1->end); (*file1)<<endl<<"Not a letter of the Latin alphabet: "<<sym;	∅

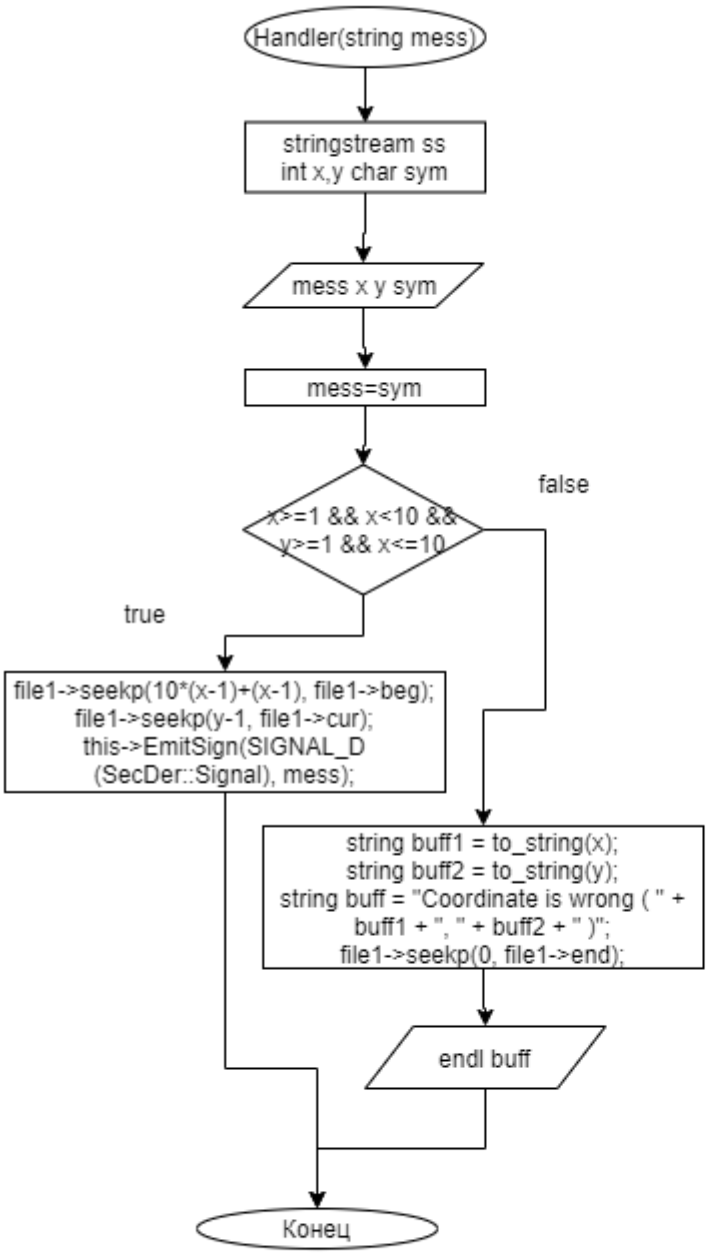
OutpFile()

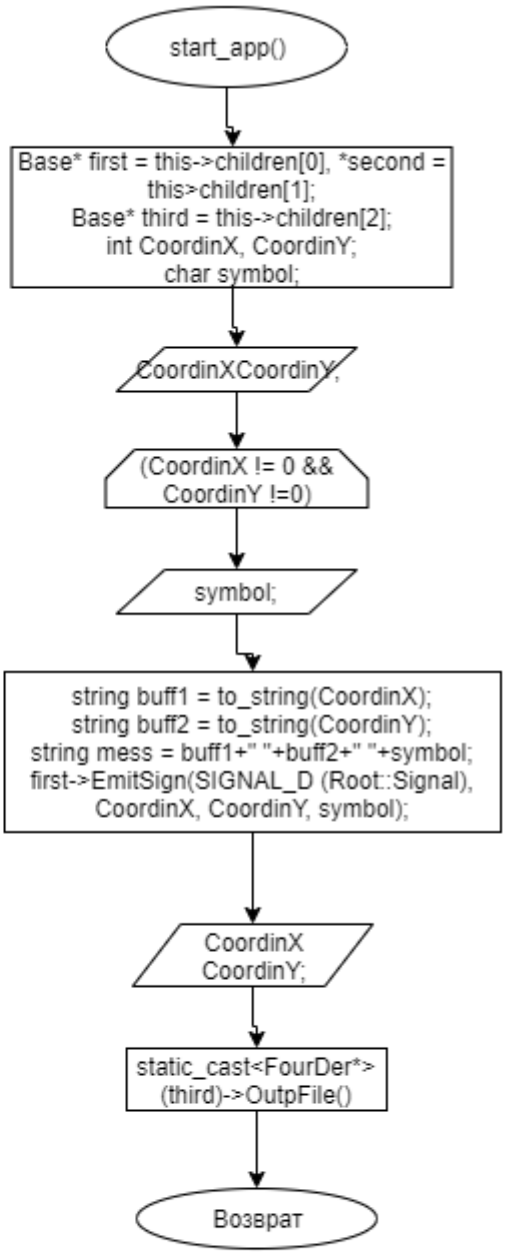
№шага	Предикат	Действие	№перехода
1		string buff; getline((*file1), buff); cout<<buff;	2
2	getline((*file1),buff)	cout<<endl<<buff;	3
		delete file1;	∅
3	getline((*file1),buff)	cout<<endl<<buff	2
	иначе		2

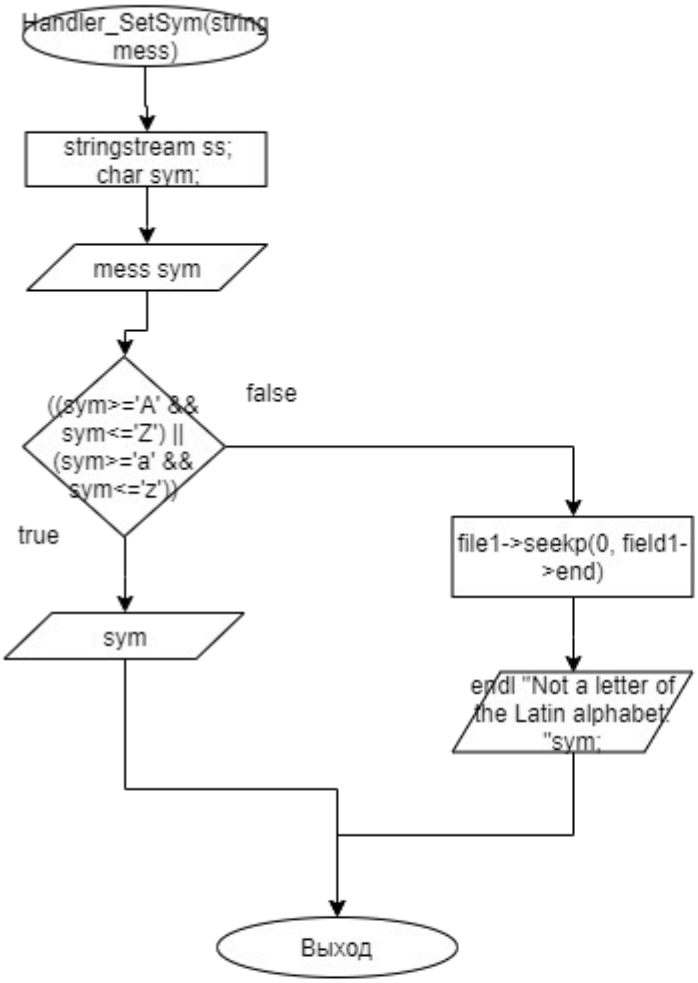
Блок-схема алгоритма

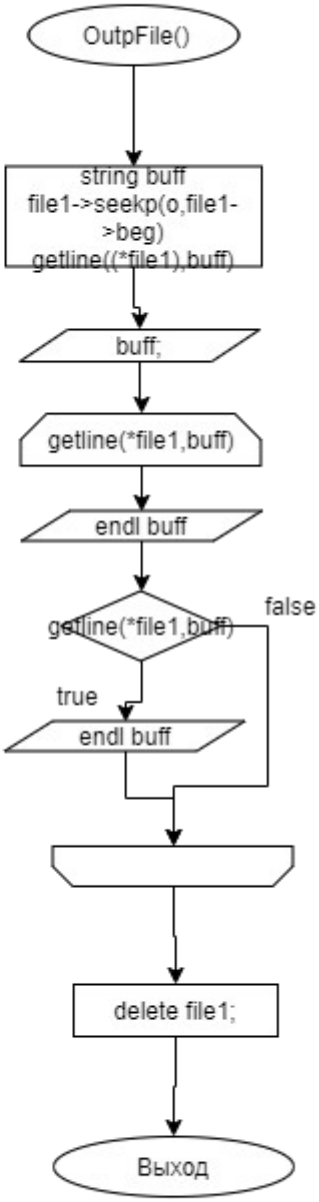

main()
Root root(nullptr)
root.input_create()
return
root.start_app()
Конец
~Base()
i=0..children.size()
delete this->children[i]
children.clear()
i=0..children.size()
delete this->connections[i]
connections.clear()
Выход
Base(Base* parent)
_parent = parent
parent!=nullptr
false
true
parent->AddChild(this)
Выход
Base(string name, Base* parent)
_parent = parent
_name = name
parent!=nullptr
false
true
parent->AddChild(this)
Выход
AddChild(Base* parent)
childrem.push_back(child)
Выход
SetName(string name)
_name = move(name)
Выход
GetName()
return _name
Возврат
SetStream(fstream* file)
file1 = file
Выход
SetStatus(int Check)
ReadCheck = Check
Выход

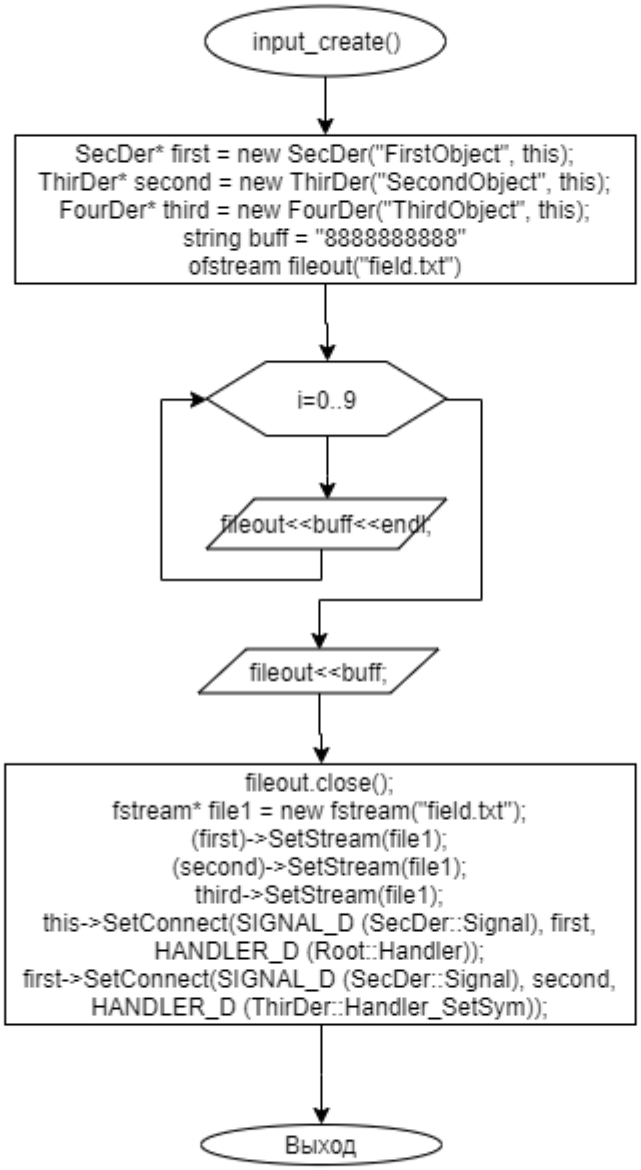

SetConnect(signal psignal, Base* pobject, handler phandler)
i=0..childrem.size()
((connections[i]->psignal==psignal) && (connections[i]->tarobject==pobject) && (connections[i]->phandler==phandler))
true
false
return
con* pntr = new con();
pntr->psignal = psignal;
pntr->tarobject = pobject;
pntr->phandler = phandler;
connections.push_back(pntr);
Возврат
DelConnect(signal psignal, Base* pobject, handler phandler)
i=0..childrem.size()
((connections[i]->psignal==psignal) && (connections[i]->tarobject==pobject) && (connections[i]->phandler==phandler))
true
false
connections.erase (connections.begin()+i);
return;
Возврат
EmitSign(signal psignal, string mess)
i=0..connections.size()
((connections[i]->psignal) == psignal)
true
false
(((connections[i])->tarobject)->*(connections[i]->phandler))(mess);
Выход


Handler(string mess)
stringstream ss
int x,y char sym
mess x y sym
mess=sym
x>=1 && x<10 &&
y>=1 && x<=10
false
true
file1->seekp(10*(x-1)+(x-1), file1->beg);
file1->seekp(y-1, file1->cur);
this->EmitSign(SIGNAL_D
(SecDer::Signal), mess);
string buff1 = to_string(x);
string buff2 = to_string(y);
string buff = "Coordinate is wrong (" +
buff1 + ", " + buff2 + ")";
file1->seekp(0, file1->end);
endl buff
Конец


start_app()
Base* first = this->children[0], *second =
this>children[1];
Base* third = this->children[2];
int CoordinX, CoordinY;
char symbol;
CoordinXCoordinY;
(CoordinX != 0 &&
CoordinY !=0)
symbol;
string buff1 = to_string(CoordinX);
string buff2 = to_string(CoordinY);
string mess = buff1+" "+buff2+" "+symbol;
first->EmitSign(SIGNAL_D (Root::Signal),
CoordinX, CoordinY, symbol);
CoordinX
CoordinY;
static_cast<FourDer*>
(third)->OutpFile()
Возврат


Handler_SetSym(string mess)
stringstream ss;
char sym;
mess sym
((sym>='A' && sym<='Z') || (sym>='a' && sym<='z'))
false
true
sym
file1->seekp(0, field1->end)
endl "Not a letter of the Latin alphabet: "sym;
Выход


OutpFile()
string buff
file1->seekp(o,file1->beg)
getline((*file1),buff)
buff;
getline(*file1,buff)
endl buff
getline(*file1,buff)
false
true
endl buff
delete file1;
Выход



Код программы

Файл Base.cpp

```
#include "Base.h"
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
//using namespace std;
Base::~Base()
{
        for (int i=0;i<children.size();i++) //удаление всех созданных объектов
        {
                delete this->children[i];
        }
        children.clear();
        for (int i=0;i<connections.size();i++) //удаление всех созданных
структур соединений
        {
                delete this->connections[i];
        }
        connections.clear();
}
Base::Base(Base* parent)
{
        _parent = parent;
        if (parent != nullptr)
        {
                parent->AddChild(this);
        }
}
Base::Base(string name, Base* parent)
{
        _parent = parent;
        _name = name;
        if (parent != nullptr)
        {
                parent->AddChild(this);
        }
}
Base* Base::GetPointTo(string name)
{
        int pos=0;
        string token;
        Base* tmp = this;
        if ((pos=name.find("//"))!=-1)
        {
                name.erase(0, 2);
                return this->FoundObject(name, true);
        }
        else
        {
                name.erase(0, 1);//удаляем первый /
                while ((pos=name.find("/"))!=-1)
                {
```

```
                        token = name.substr(0, pos);
                        if (token==this->GetName())
                        {
                                tmp = this;
                        }
                        else
                        {
                                tmp = tmp->FoundObject(token, false);
                        }
                        if (tmp==nullptr)
                        {
                                return nullptr;
                        }
                        else
                        {
                                name.erase(0, pos+1);
                        }
                }
                if (name == this->GetName())
                {
                        return this;
                }
                else
                {
                        return tmp->FoundObject(name, false);
                }
        }
}
string Base::GetName()
{
        return _name;
}
Base* Base::FoundObject(string name, bool flag)
{
        Base* tmp = nullptr;
        for (int i=0;i<children.size();i++)
        {
                if (children[i]->GetName() == name)
                {
                        return children[i];
                }
                else
                {
                        if (flag)
                        {
                                if ((tmp = children[i]->FoundObject(name,
true)) != nullptr)
                                {
                                        return tmp;
                                }
                        }
                }
        }
        return nullptr;
}
void Base::SetConnect(signal psignal, Base *pobject, handler phandler) //метод
установки связи
{
        for (int i =0; i<connections.size();i++) //поиск уже существующего
соединения в векторе соединений
```

```
        {
                if ((connections[i]->psignal==psignal) && (connections[i]-
>tarobject==pobject) && (connections[i]->phandler==phandler))
                {
                        return; //если есть совпадение, то не создаем
повторную связь
                }
        }
        con* pntr = new con(); //указатель на структуру
        pntr->psignal = psignal; //заполнение полей структуры
        pntr->tarobject = pobject;
        pntr->phandler = phandler;
        connections.push_back(pntr); //добавление заданной структуры в вектор
}
void Base::DelConnect(signal psignal, Base *pobject, handler phandler) //метод
удаления свзяи
{
        for (int i =0; i<connections.size();i++) //поиск соединения в векторе
соединений
        {
                if (((connections[i]->psignal)==psignal) && ((connections[i]-
>tarobject)==pobject) && ((connections[i]->phandler)==phandler))
                {
                        connections.erase(connections.begin()+i); //удаление
нужного соединения
                        return;
                }
        }
}
void Base::EmitSign(signal psignal,string mess) //метод выдачи сигнала
{
        for (int i=0;i<connections.size();i++) //поиск нужного соединения в
векторе соединений
        {
                if ((connections[i]->psignal) == psignal)
                {
                        (((connections[i])->tarobject)->*(connections[i]-
>phandler))(mess); //вызов метода обработчика целевого объекта
                }
        }
}
void Base::AddChild(Base* child)
{
        children.push_back(child);
}
void Base::SetStatus(int Check)
{
        ReadCheck = Check;
}
void Base::SetName(string name)
{
        _name = move(name);
}
void Base::Print(int counter)
{
        counter = counter + 4;
```

```
        int buff = counter;
        for (int i=0; i<children.size();i++)
        {
                cout << endl;
                while (buff != 0)
                {
                        cout<<" ";
                        buff--;
                }
                cout<<children[i]->GetName();
                children[i]->Print(counter);
                buff = counter;
        }
}
```

Файл Base.h

```
#ifndef BASE_H
#define BASE_H
#include <string>
#include <vector>
#include <iostream>
#include <fstream>
using namespace std;
#define SIGNAL_D(func) (signal) (&func)
#define HANDLER_D(handler_f) (handler) (&handler_f)
class Base
{
public:
        typedef void (Base::*signal) (string); //указатель на метод сигнала
        typedef void (Base::*handler) (string); //указатель на метод
обработчика
        struct con //структура для соединений
        {
                signal psignal; //указ. на метод сигнала
                Base* tarobject;//указ на объект целевой
                handler phandler;//указ на метод обработчика
        };
        Base* _parent;
        string _name;
        int ReadCheck;
        vector <Base*> children; //вектор наследников
        vector <con*> connections;//вектор соединений

public:
        Base (Base* parent=nullptr); //конструктор1
        Base (string name, Base* parent = nullptr); //конструктор 2
        string GetName();
        void SetName(string name);
        void AddChild(Base* child);
        void SetStatus(int Check);
        void SetConnect(signal psignal, Base* pobject, handler phandler);
//метод создания соединения
        void DelConnect(signal psignal, Base* pobject, handler phandler);
```

```
//метод удаления соединения
        void EmitSign(signal psignal,string mess); //метод выдачи сигнала
        Base* GetPointTo(string name); //методы из предыдущиз КР. В данной не
используются
        Base* FoundObject(string name, bool flag);
        void Print(int counter);
        ~Base();
};

#endif
```

Файл FourDer.cpp

```
#include "Base.h"
#include "FourDer.h"
#include <string>
#include <stdlib.h>
#include <iostream>
#include <vector>
#include <sstream>
using namespace std;
void FourDer::OutpFile() //метод вывода сожержимого файла
{
        string buff;
        file1->seekp(0, file1->beg);
        getline((*file1), buff); //считываем первую строку
        cout<<buff;
        while(getline((*file1),buff)) //считываем строку
        {
                cout<<endl<<buff;
                if (getline((*file1), buff)) //если читается, то печатаем
считанную с пробелом
                {
                        cout<<endl<<buff;
                }
        }
        delete file1; //удаление указателя на файловый поток
}
```

Файл FourDer.h

```
#ifndef FOURDER_H
#define FOURDER_H
#include <string>
```

```
#include <stdlib.h>
#include <iostream>
#include <vector>
#include <sstream>
class FourDer : public Base
{
private:
        fstream* file1; //свойство для указателя на файловый поток
public:
        FourDer(string name, Base* parent) : Base (name, parent) { };
        void SetStream(fstream* file) //установка файлового потока
        {
                file1=file;
        }
        void OutpFile(); //метод вывода содержимого файла
};
#endif
```

Файл main.cpp

```
#include <stdlib.h>
#include <stdio.h>
#include <iostream>
#include "Root.h"
using namespace std;
int main()
{
        Root root(nullptr);
        root.input_create();
        return root.start_app();
}
```

Файл Root.cpp

```
#include "Root.h"
#include "SecDer.h"
#include "Base.h"
#include "ThirDer.h"
#include "FourDer.h"
#include <string>
#include <iostream>
#include <fstream>
#include <vector>
void Root::input_create()
{
        SecDer* first = new SecDer("FirstObject", this); //создаем три объекта
1 для выдачи сигнала
        ThirDer* second = new ThirDer("SecondObject", this); //для записи в
```

```
файл по координате
        FourDer* third = new FourDer("ThirdObject", this);  //для вывода файла
        string buff = "8888888888";
        ofstream fileout("field.txt");
        for (int i=0;i<9;i++)
        {
                fileout<<buff<<endl; //заполнение файла
        }
        fileout<<buff;
        fileout.close();
        fstream* file1 = new fstream("field.txt"); //создание файлового потока
        (first)->SetStream(file1); //установка файлового потока для объектов
        (second)->SetStream(file1);
        third->SetStream(file1);
        this->SetConnect(SIGNAL_D (Root::Signal), first, HANDLER_D
(SecDer::Handler));//установка связи между корневым и 1 объектом
        first->SetConnect(SIGNAL_D (SecDer::Signal), second, HANDLER_D
(ThirDer::Handler_SetSym));//установка связи между 1 и 2 объектом
}
int Root::start_app()
{
        Base* first = this->children[0], *second = this->children[1];
        Base* third = this->children[2]; //получение указателя на созданные
объекты
        int CoordinX, CoordinY;
        char symbol;
        cin>>CoordinX>>CoordinY; //считывание координат
        while (CoordinX != 0 && CoordinY != 0)
        {
                cin>>symbol;
                string buff1 = to_string(CoordinX);
                string buff2 = to_string(CoordinY);
                string mess = buff1+" "+buff2+" "+symbol; //формирование
строки для передачи
                this->EmitSign(SIGNAL_D (Root::Signal), mess); //выдача
сигнала объектом
                cin>>CoordinX>>CoordinY;
        }
        static_cast<FourDer*>(third)->OutpFile(); //вывод файла третьим
объектом
        return 0;
}
```

Файл Root.h

```
#ifndef ROOT_H
#define ROOT_H
#include <string>
#include <iostream>
#include <vector>
```

```
#include <fstream>
#include "Base.h"
//using namespace std;
class Root : public Base
{
public:
        Root(Base* parent=nullptr) : Base(parent){};
        void input_create(); //метод ввода
        int start_app(); //метод отработки приложения
        void Signal(string &mess) {};
        //void Handler(string mess);
};
#endif
```

Файл SecDer.h

```
#ifndef SECDER_H
#define SECDER_H
#include <vector>
#include <string>
#include <iostream>
#include <fstream>
#include <sstream>
#include "ThirDer.h"
class SecDer : public Base
{
private:
        fstream* file1; //свойство для указателя на файловый поток
public:
        SecDer(string name, Base* parent) : Base (name, parent) { };
        void Signal (string &mess){ }; //метод сигнала объекта
        void SetStream(fstream* file) //установка файлового потока
        {
                file1 = file;
        }
        void Handler(string mess) //метод обработчика объекта для установки
координаты
        {
                stringstream ss; //строковый поток
                int x,y;
                char sym;
                ss<<mess; //запись пришедшего сообщения
                ss>>x; //считывание координат и символа
                ss>>y;
                ss>>sym;
                mess = sym; //в сообщение записываем символ
                if ((x>=1) && (x<=10) && (y>=1) && (y<=10)) //проверка
координат
                {
                        //file1->seekp(0, file1->beg);
                        file1->seekp(10*(x-1)+(x-1), file1->beg); //установка
позиции
                        file1->seekp(y-1, file1->cur);
                        this->EmitSign(SIGNAL_D (SecDer::Signal), mess);
```

```
//выдача сигнала от данного объекта 2-му, для дальнейшей записи символа
                }
                else  //если координаты не соответствуют
                {
                        string buff1 = to_string(x);
                        string buff2 = to_string(y);
                        string buff = "Coordinate is wrong ( " + buff1 + ", "
+ buff2 + " )";  //формирование строки для записи в файл
                        file1->seekp(0, file1->end); //установка позиции для
записи
                        (*file1)<<endl<<buff; //запись
                }
        }
};
#endif
```

Файл ThirDer.cpp

```
#include "Base.h"
#include "ThirDer.h"
#include <string>
#include <stdlib.h>
#include <iostream>
#include <vector>
#include <sstream>
using namespace std;
void ThirDer::Handler_SetSym(string mess) //метод обработчика для записи в
установленную позицию
{
        stringstream ss; //строковый поток
        char sym;
        ss<<mess; //записываем в строковый поток сообщ.
        ss>>sym; //считываем симвод
                if ((sym>='A' && sym<='Z') || (sym>='a' && sym<='z'))
//проверка на соответствие символа алфавиту
                {
                        (*file1)<<sym; //запись в позицию
                }
                else //если не соответсвует
                {
                        file1->seekp(0, file1->end);
                        (*file1)<<endl<<"Not a letter of the Latin alphabet:
"<<sym; //записываем в конец файла строку
                }
}
```

Файл ThirDer.h

```
#ifndef THIRDER_H
#define THIRDER_H
#include <string>
#include <stdlib.h>
#include <iostream>
#include <vector>
#include <sstream>
class ThirDer : public Base
{
private:
        fstream* file1; //свойство для указателя на файловый поток
public:
        ThirDer(string name, Base* parent) : Base (name, parent) { };
        void SetStream(fstream* file)//установка файлового потока
        {
                file1=file;
        }
        void Signal(string &mess) { };
        void Handler_SetSym(string mess);
        //void OutpFile();
};
#endif
```

Тестирование

Входные данные	Ожидаемые выходные данные	Фактические выходные данные

12 9 a 13 10 0 1 1 h 2 2 e 3 3 l 4 4 l 5 5 0 6 6 ! 12 12 7 10 1 W 10 2 O 10 3 r 10 4 L 10 5 D 0 0	h888888888 8e88888888 88l8888888 888l888888 8888888888 8888888888 8888888888 8888888888 8888888888 WOrLD88888 Coordinate is wrong (12, 9) Coordinate is wrong (13, 10) Not a letter of the Latin alphabet: 0 Not a letter of the Latin alphabet: ! Coordinate is wrong (12, 12)	h888888888 8e88888888 88l8888888 888l888888 8888888888 8888888888 8888888888 8888888888 8888888888 WOrLD88888 Coordinate is wrong (12, 9) Coordinate is wrong (13, 10) Not a letter of the Latin alphabet: 0 Not a letter of the Latin alphabet: ! Coordinate is wrong (12, 12)